

ORCA 2.0 – Phase 0 Build Brief

For the agent building the foundation · August 2026

This brief is self-contained. You do not need the conversation that produced it. Where it points at a companion document, read that document before building the thing it describes.

1 · What you are building, and why it is not a service

ORCA is a gate-automation platform for logistics facilities: cameras read truck plates, customer-designed processes orchestrate devices and external systems, work automation cannot finish goes to a clerk, carriers pre-announce visits. It is being rebuilt as **seven Java 25 / Spring Boot services on Microsoft SQL Server**, deployed as one repository.

Phase 0 builds none of that. It builds the foundation the seven services stand on: five shared primitives, the checks that enforce them, migrations, CI, and a local stack.

Why this is a separate phase. Every correctness guarantee in the architecture — nothing lost on restart, nothing half-applied, exactly one visit per truck, no query escaping its scope — is a property of these five primitives. If four developers each start a service, each will build their own version of all five in their first week, and the platform will have four subtly different implementations of its own guarantees. That is the exact defect class this rebuild exists to remove.

In this repository with you, under `docs/` : `ORCA_ARCHITECTURE.md` (the design), `ORCA_OPEN_QUESTIONS_REGISTER.md` (what is deliberately unsettled — consult it before assuming something is missing by accident), and this brief.

Read before you start: `docs/ORCA_ARCHITECTURE.md` — particularly **§B4** (the five communication mechanisms), **§B8** (conventions), and **§B10** (what the architecture guarantees). **§B10** is the acceptance criteria for this phase, expressed as properties.

2 · Ground rules

These are not style preferences. Each one prevents a specific failure.

RULE	WHY
Never invent a resolution to an unspecified question. Leave it unbuilt, and say so in your final report	A plausible guess written as working code is far harder to find later than a gap. If the architecture is silent, that silence is information
platform/ holds primitives, never domain. If a class there knows what a visit, lane, ticket or driver is, it belongs in a service	A shared module that accumulates domain logic becomes the thing everything depends on and nobody can change
A guarantee needs a test that proves the property , not one that exercises the code path	"The outbox writes a row" is not a test. "Killing the process between the two writes leaves neither" is
Build checks land with the code they govern, never after	A check added afterwards certifies whatever was written rather than constraining it
No dependency beyond §3 without recording why in the version catalog as a comment	§3 lists what was deliberately excluded. Re-adding one silently reverses a signed decision
No service business logic in this phase	Services get a skeleton, a health endpoint, and their module boundaries. Nothing else
Microsoft SQL Server is the only database. There is no PostgreSQL deployment	Advice elsewhere that assumes PostgreSQL — particularly about row-level security — does not apply

3 • The repository

The root project is generated from Spring Initializr and is **already committed**: Gradle · Kotlin DSL · Java 25 · Spring Boot 4.0.7 · group `com.lynxis` · artifact `orca` · package `com.lynxis.orca` · jar · YAML configuration.

Java 25 and Spring Boot 4.0.7 are settled. Java 25 is the current long-term-support release — a non-LTS JVM is unsuitable for an appliance that runs unattended at a customer site for years. Spring Boot 4.0.7 was chosen over 4.1.0 because a patched minor carries fewer surprises than the first release of a new one, and Flowable's Spring Boot 4 support was confirmed before committing.

```
orca/
├─ settings.gradle.kts
├─ build.gradle.kts
├─ gradle/libs.versions.toml
├─ platform/
├─ │ outbox/ lease/ scope/ idempotency/ web/
├─ services/ # each owns its schema, migrations AND contract
├─ │ orca-core/ orca-runtime/ orca-edge/
├─ │ orca-portal/ orca-sync/ orca-fleet/
├─ │ orca-media/ # README only – NOT a Gradle module
├─ build-checks/ # ArchUnit rules; fail the build, not the review
├─ deploy/
├─ │ bootstrap/ # schemas, logins, grants – once, before any service
├─ │ docker-compose.yml
├─ │ keycloak/
```

Dependencies already in the base project: Spring Web · Spring Data JPA · Validation · Spring Security · OAuth2 Resource Server · Actuator · **MS SQL Server Driver** · Flyway · Lombok · Testcontainers.

Add only where noted: `springdoc-openapi-starter-webmvc-ui` (all services) · `archunit` (build-checks) · `flowable-spring-boot-starter-process` (orca-runtime, **process engine only**) · `spring-boot-starter-websocket` (orca-runtime) · Netty or Spring Integration (orca-edge).

Deliberately excluded — do not add: Spring for Apache Kafka · Spring Cloud (Config, Gateway, Eureka) · Redis or any cache starter · PostgreSQL driver · Spring Session.

Decisions already taken – do not re-decide these

Lombok	In. Use it consistently
Code formatter	None for now. Do not add Spotless, google-java-format or a checkstyle config
Commit convention	Free-form. Commit once per work package, with a message saying what landed
Local secrets	<code>.env</code> is gitignored . Commit a <code>.env.example</code> with working local values and a header stating it is for local development only. These are fixtures, not secrets – a developer should clone, copy one file, and <code>docker compose up</code>

Package layout

Every module uses the same three packages, and the module names come from the architecture so that a developer reading §C2 and opening the code sees the same names.

```
com.lynxis.orca
├─ platform
├─ │ outbox · lease · scope · idempotency · web
├─ runtime
├─ │ execution · workitem · integration · notify · readmodel
├─ core · edge · portal · sync · fleet
```

⚠️ Only `runtime` has **architecture-defined modules**. §C2 names those five and the module wall depends on them, so create them.

`core`, `edge`, `portal`, `sync` and `fleet` are flat in this phase — the service package with `api`, `domain` and `persistence` directly beneath it. The architecture does not decompose them into modules, and **inventing a decomposition here would be exactly the failure rule §2 prohibits**. When their modules are defined, they can be added; a flat package is trivially split, and a wrong split is not.

Inside every module:

```
com.lynxis.orca.<service>.<module>
├─ api          controllers and DTOs
├─ domain       entities and domain services
└─ persistence  repositories
```

This split is what makes the module-wall check expressible. The rule is "no module may reference another module's `persistence` package" — one ArchUnit statement, and it is the wall §B5 and §C2 depend on. Keep the three packages even where a module currently has little in one of them.

What the base project is, and what you make of it

You are given **one** Spring Initializr project. Turn it into a **Gradle multi-project**: the generated root becomes the parent, and you create the modules beneath it.

Twelve Gradle modules in total — five under `platform/`, six services, and `build-checks`. `orca-media` is a README, not a module. **Six bootable applications**.

Every service module is its own bootable Spring Boot application — its own `main` class, its own configuration, its own image. They are independently deployable and share one version catalog and one repository. That is ADR-014, and it is why the build checks can see across all seven.

4 · Work packages, in order

Package 1 – Build wiring

Register every module in `settings.gradle.kts`. The root build sets the Java 25 toolchain and one version catalog in `gradle/libs.versions.toml`; every module declares only its own dependencies.

The root project is an aggregator and holds no code. Spring Initializr can only generate a single application, so it produced a bootable one at the root — `src/` and `OrcaApplication.java`. **Both are removed in this package**.

	GENERATED STATE	REQUIRED STATE
Root <code>src/</code>	A bootable application	Deleted
Spring Boot plugin	Applied at the root	Declared at the root with <code>apply false</code> , applied in each service module
Root <code>build.gradle.kts</code>	Builds a jar	Aggregator only – toolchain, version catalog, shared configuration
Bootable applications	One, at the root	Seven , one per service module

Each service owns its own entry point, for example `services/orca-core/src/main/java/com/lynxis/orca/core/CoreApplication.java`.

⚠️ **Do not keep the root application "just in case."** Leaving it produces an eighth Spring Boot application that does nothing, claims the default port, and misleads everyone who clones the repository.

Before deleting it, keep what is useful: the generated `TestcontainersConfiguration.java` shows this Spring Boot version's Testcontainers wiring. Move that pattern into the shared test configuration rather than discarding it.

A note on this Spring Boot version's starter names. The starters were renamed in Boot 4 — it is `spring-boot-starter-webmvc`, not `spring-boot-starter-web`, and each starter has a paired `-test` artifact (`spring-boot-starter-data-jpa-test`, `spring-boot-starter-webmvc-test`) rather than one blanket `spring-boot-starter-test`. The committed `build.gradle.kts` has the correct names — **copy from it rather than from memory or from older examples**.

Done when `./gradlew build` succeeds from a clean clone with nothing installed but a JDK, and no bootable application remains at the root.

Package 2 – Local stack

`deploy/docker-compose.yml` with **SQL Server** and **Keycloak**. Keycloak seeded from a realm import file in the repository: one realm, one client per service, service-account grants for service-to-service calls.

Done when `docker compose up` yields a working database and identity server, and a service can obtain a token and validate it **by signature, locally** — no call-out per request.

Package 3 – Migrations

Each service owns its own migrations, in its own module. A service owns its schema — that is ADR-004, enforced by database credentials — and a schema whose definition lives outside the service that owns it is not owned by it. Migrations sit under each service's own resources:

```
services/orca-core/src/main/resources/db/migration/  
services/orca-runtime/src/main/resources/db/migration/  
...
```

Each service runs Flyway against **its own schema only**, on its own startup, with its own credentials. No service can migrate another's schema, for the same reason no service can write another's tables.

One thing is not per-service, because a service cannot do it for itself. Creating the schemas, the per-service database logins, and the grants that restrict each login to its own schema are privileged operations — a service authenticating as `orca_core` cannot create the `orca_core` login. That bootstrap lives in `deploy/bootstrap/` as versioned SQL, and runs once against a fresh database before any service starts.

The ordering dependency is a deployment concern, not a build one. `orca-core` publishes the read-only views other services consume, so core must have migrated before a service that reads them starts. Handle it where it belongs:

- Deployment order — core migrates and starts first.
- **A startup readiness check in each dependent service:** if a view it requires is absent, fail immediately with a message naming the missing view. A service that starts and then fails on the first query is far harder to diagnose than one that refuses to start and says why.

⚠ **Do not create a shared migrations module to enforce the ordering.** It would centralise schema definition away from the services that own it, and it does not remove the dependency — it only hides it behind one runner.

Package 4 – The five primitives

The substance of this phase. Specifications in §5.

Package 4b – Contract-first API generation

This is ADR-014 made enforceable. The OpenAPI document is the source of truth; the Java API layer is generated from it. **Each service owns its contract**, alongside the schema and migrations it already owns:

```
services/orca-core/src/main/resources/openapi/orca-core.yaml  
services/orca-runtime/src/main/resources/openapi/orca-runtime.yaml  
...
```

Keeping it in the service's own resources means it ships inside the jar, so a running service can serve its own specification rather than depending on a copy kept elsewhere.

The shared components live with the primitive that implements them:

```
platform/web/src/main/resources/openapi/_shared.yaml
```

The response envelope, the error object and pagination are **defined by `platform/web` and implemented by it**. Putting the schema anywhere else separates the contract from the code that fulfils it. Every service's contract references this file.

⚠️ **One wrinkle to solve rather than avoid:** a cross-module `$ref` needs the generator to resolve a path outside the service's own module. Configure it; do not work around it by copying `_shared.yaml` into each service. Seven copies of the envelope schema is the failure this file exists to prevent.

Set up `openapi-generator-gradle-plugin` with the `spring` generator, per service:

SETTING	VALUE	WHY
<code>interfaceOnly</code>	true	Generate the API interface and its DTOs. Controllers are hand-written and <i>implement</i> the interface – so a contract change breaks the build until the implementation matches
<code>useJakartaEe</code>	true	Spring Boot 4
Output directory	<code>build/generated</code> – never committed	Generated sources in version control drift from their spec and nobody notices
<code>contracts/_shared.yaml</code>	The response envelope, the error object, error codes, pagination metadata – referenced by all seven contracts	One definition. Seven services each defining their own "error" schema is how P5's guarantee decays into a convention

springdoc serves the UI; it does not generate the spec. Point Swagger UI at the checked-in YAML, or serve the file statically and drop springdoc. Do not let it introspect the code — code-first and contract-first together means two sources of truth, and they will disagree.

⚠️ **Smoke-test the generator against Spring Boot 4 in Package 1, before seven services depend on it.** Generate one trivial contract and compile it. The `spring` generator has historically lagged Spring major versions; discovering a gap here is cheap, discovering it in Phase 1 is not. **If it cannot emit Boot 4-compatible code, stop and report it** — do not hand-write the API layer and carry on, because that silently reverses ADR-014.

Done when one service's contract generates an interface, a hand-written controller implements it, the build fails if the contract and the controller disagree, and all seven contracts share the envelope from `_shared.yaml`.

Package 5 – Build checks

ArchUnit rules in `build-checks/`, wired so they fail `./gradlew build`:

CHECK	FAILS WHEN
Platform purity	A class in <code>platform/</code> references a domain type
Module walls	A service imports another service's internals; a module reads another module's tables
Scope seam	A query is constructed outside the seam
Error envelope	A controller returns a shape other than the envelope
Retention class	A table declared as traffic-growing has no retention class

⚠️ Write each check **with** the primitive it governs, in the same commit where practical.

Package 6 – CI

Build → unit tests → integration tests on Testcontainers against real SQL Server → all build checks → coverage report.

Done when a pull request cannot merge with a failing check, and the pipeline completes in under ten minutes.

5 • The five primitives

What `platform/` is, and what it is not

It is not a utility module. It is the five mechanisms every service uses to be correct, built once so that "correct" means the same thing in all six.

Each primitive exists because the same hard problem appears in every service, and solving it six times produces six subtly different answers. Concretely:

PRIMITIVE	THE PROBLEM IT SOLVES, IN ONE SENTENCE	WHAT A SERVICE DOES WITH IT
outbox	A service must tell other services that something happened, without a message broker, and without the fact and the notification ever disagreeing	<code>orca-runtime</code> finishes a visit and writes the visit and its outbox row in one transaction. <code>orca-portal</code> is told, once, in order
lease	Some work may only be done by one instance at a time, and the instance holding that right may die at any moment	<code>orca-edge</code> polls a lane's camera. Exactly one instance owns that lane; if it stops, another takes over, and the dead one cannot write afterwards
scope	Every query must be limited to the data the caller may see, and forgetting once is a breach	A repository asks for work items. The scope predicate is applied by the seam, not typed into the query – and a query written outside the seam does not compile
idempotency	The same request will arrive twice – retries, timeouts, redelivery – and must have the effect of one	<code>orca-runtime</code> sends a gate command; the response is lost; it retries. The barrier rises once, and the retry receives the first outcome rather than an error
web	Callers need one response shape and machine-readable errors, and jobs that run without a user still need an identity	Every controller returns the same envelope. The outbox relay, which no user invoked, runs under an explicit system identity rather than none

The rule, stated as a test you can apply to any class: if it names a **visit**, a **lane**, a **ticket**, a **driver** or a **truck**, it does not belong in `platform/`. The primitives know about transactions, leases, keys, scopes and HTTP. They do not know what business this is.

Why this is Phase 0 and not "later, when we need it". Retrofitting is the expensive direction. Once six services have each written their own outbox and their own scoping, unifying them means touching every query in the product — which is precisely the position the current system is in, with roughly 816 hand-written scope conditions and no single place to fix them.

Each primitive below is an interface, **one** implementation, and tests that prove the property.

P1 • Transactional outbox and relay – `platform/outbox`

Replaces a message broker. There is no broker inside a site.

Shape. An outbox table per owning schema: a monotonic sequence, an **ordering key** (facts are ordered per key, never globally), an event type, a payload, a creation timestamp — plus per-consumer delivery tracking, because a fact is acknowledged by each registered consumer separately.

Behaviour.

- The business fact and its outbox row are written in **one transaction**. Neither can exist without the other.
- A relay claims work with a **skip-locked read** so two instances never contend for the same row.
- Delivery is **at-least-once, ordered per ordering key**. Consumers must be idempotent — that is P4's job, not this one's.
- A row is deletable only when **every registered consumer has acknowledged it**.

Tests that prove it.

1. Kill the process between the fact write and the outbox write — assert **neither** exists.
2. Deliver the same batch twice — assert **one** effect.
3. Two relay instances against one outbox — assert no row is delivered twice and none is skipped.

4. Hold one consumer back — assert nothing it has not acknowledged is deleted.

P2 • Lease with fence token – platform/lease

Anything only one instance may do at a time.

Shape. A `service_lease` table keyed `(service, lease_name)`, holding the holder identity, acquisition and renewal timestamps, an expiry, and a **fence token**. No `site_id`, no audit columns, no soft delete — this is process-coordination state, not tenant data.

Behaviour.

- Acquisition is a **conditional update**, not a read-then-write.
- The **fence token increases every time the lease changes hands**.
- A holder presents its token with any write the lease protects. **A write carrying a stale token is rejected.**
- **Expiry is judged by the database clock**, never an instance's clock.

Tests that prove it.

1. Two instances race to acquire — exactly one wins.
2. Expire the lease mid-operation, then attempt the write — assert **rejected**, not silently applied.
3. Assert the token strictly increases across handovers.

P3 • The scope seam – platform/scope

One place where a query acquires its scope predicate, and no way around it.

What the architecture requires (§B6): *"Scope is enforced in one place, applied by construction, with a build-time check that fails when a query bypasses it. No query carries its own scoping condition."*

The requirement is settled; the mechanism is not. Build the seam as an interface with a single implementation that applies the predicate, and the ArchUnit rule that makes bypassing it a build failure. **Do not** implement database row-level security in this phase — that choice belongs to the security design and has a named owner.

⚠ **If SQL Server row-level security is ever chosen, the connection pool is the trap, not the SQL.**

`sp_set_session_context` is session-scoped and a JDBC pool hands out shared connections; setting the key on a pooled connection leaves it set for whoever borrows it next. Unpinned, that is **worse than no RLS** — every request is silently scoped to an earlier request's tenant and it looks like it is working. Record this in the module's documentation so whoever implements it later does not rediscover it in production.

Tests that prove it.

1. A repository method building a query outside the seam **fails the build**.
2. A query through the seam with no scope set returns **zero rows, never all rows**.

P4 • Idempotency record – platform/idempotency

Shape. A recorded-key table: the key, the operation, the recorded outcome, a timestamp.

Behaviour.

- The same key applied twice has the effect of once.
- The second call returns the **recorded outcome** — never a bare "duplicate" with no result. A caller that retried because it never saw the first answer needs the answer, not an error.
- An operation still in flight is reported as **in progress**, and that is **not a terminal state** — the caller keeps waiting.

Tests that prove it.

1. Replay a completed operation — assert the recorded outcome is returned.
2. Replay an in-flight operation — assert *in progress*, and that it is not treated as an answer.
3. Concurrent first-attempts with the same key — exactly one executes.

P5 • Web envelope and system context – platform/web

Shape. One response envelope for every service: status, machine-readable code, message, data, an errors array, pagination metadata, and a request identifier.

Behaviour.

- Every distinct error has a distinct code a caller can branch on without reading the message.
- Internal detail never reaches a caller — no stack traces, no exception text, no schema names.
- Every entry point that runs without a user — a relay, a scheduled job, a reconciler — enters an explicit system context. No path runs with no identity at all.

Tests that prove it.

1. Assert no error path serialises an exception or a schema name.
2. Assert every scheduled entry point sets the system context.
3. Assert two distinct failures return two distinct codes.

6 • Services in this phase

Each of the seven gets: a module, a Spring Boot application class, a health endpoint, its Flyway folder, its OpenAPI stub in `contracts/`, and its module package boundaries with the ArchUnit rule enforcing them.

Nothing else. No entities beyond what a primitive needs, no controllers beyond health, no business logic.

`orca-media` is a placeholder only — it is inherited and not built here.

7 • Verification – run these, do not assume them

Every item below is a command you execute and whose output you keep. Writing a test is not evidence that it passes. Before you report, run all of it from a clean state and record what happened.

#	RUN	EXPECTED
1	<code>git clean -xdf && ./gradlew build</code>	Succeeds from a clean tree with nothing installed but a JDK
2	<code>docker compose -f deploy/docker-compose.yml up -d</code>	SQL Server and Keycloak reach healthy
3	<code>./gradlew flywayMigrate</code> (or the equivalent task)	All schemas created, core first
4	The same migration command, a second time	A no-op. No error, no duplicate objects
5	<code>./gradlew test</code>	Every primitive's property tests pass, including the kill-mid-transaction and lease-expiry cases
6	<code>./gradlew integrationTest</code>	Testcontainers tests pass against real SQL Server
7	<code>./gradlew check</code>	All five ArchUnit rules pass
8	Deliberately violate each of the five build checks, one at a time	Each violation fails the build , with a message that names what is wrong
9	Boot each of the seven services	Each starts and answers on its health endpoint
10	Obtain a token from Keycloak and call a secured endpoint	Accepted. Then call it with a tampered token – rejected

⚠ **Item 8 is the one that is easy to skip and the most important.** A check that has never been seen to fail may not be wired in at all. Prove each one by breaking it, then revert the break.

If any item cannot be run — Docker unavailable, no network, a tool that will not install — say so explicitly in your report and name which items are unverified. Do not describe an unrun command as passing.

8 • When you are blocked

You will hit at least one of these. None is a reason to improvise.

SITUATION	WHAT TO DO
A tool or version does not support Spring Boot 4	Stop, report it, and name what it blocks. Do not hand-roll a replacement – that silently reverses a signed decision
The architecture is silent on something you need	Check ORCA_OPEN_QUESTIONS_REGISTER.md . If it is there, it is deliberately open: leave it unbuilt. If it is not, report it as a gap
Two documents disagree	The architecture wins over this brief. Report the disagreement
A test is hard to write	That is usually the design telling you something. Report it rather than weakening the test to make it pass
Docker will not start SQL Server	Almost always the SA password – it needs 8+ characters with upper, lower, digit and symbol

9 • Your final report

State plainly, and keep it short enough to be read:

- What you built, package by package.
- The verification table from §7, with the actual result of each item — including anything you could not run.
- What you could not build, and why — especially anything the architecture left unspecified. Do not fill those gaps; name them.
- Every decision you made that the architecture did not dictate, with the reasoning. These are what the team needs to review.
- Anything in the architecture that appears wrong or self-contradictory. Report it; do not silently correct it.

⚠ A gap reported is worth more than a gap filled with a guess. The team can act on the first and will not find the second until it fails.